

24. L-атрибутные грамматики. Вычисление атрибутов при обходе в глубину. Схемы трансляции.

L-атрибутная грамматика

Определение

Атрибутная грамматика называется *L-атрибутой*, если для любого правила вывода $A \rightarrow X_1 \dots X_n$ и любого $i = 1, \dots, n$ каждый наследуемый атрибут символа X_i из правой части зависит только от атрибутов символов $X_1, \dots, X_{i-1}$ и наследуемых атрибутов символа A . В частности, любая S-атрибутная грамматика является L-атрибутой.

Обход в глубину дерева вывода назовем *стандартным*, если сыновья любого узла просматриваются слева направо (т. е. если в данном узле применялось правило $A \rightarrow X_1 X_2 \dots X_n$, то его сыновья, помеченные $X_1, X_2, \dots, X_n$, просматриваются именно в этом порядке). Такой порядок просмотра является и порядком создания узлов дерева при нисходящем анализе.

Ничего не сказано про синтезируемые атрибуты, так как на них нельзя ввести никаких ограничений (они могут быть, могут не быть)

Можем наследоваться от любых (и синтезируемых, и наследуемых) атрибутов левых братьев и только от наследуемых атрибутов отца

Вычисление атрибутов в L-атрибутой грамматике

Предложение

Приведенная атрибутная грамматика является L-атрибутой тогда и только тогда, когда для любой синтаксически корректной цепочки все атрибуты грамматических символов могут быть вычислены стандартным обходом в глубину дерева вывода.

Доказательство.

I. $\Rightarrow$

DFVISIT (узел n)

1. для каждого сына m узла n
2. вычислить наследуемые атрибуты узла m
3. DFVISIT(m)
4. вычислить синтезируемые атрибуты узла n

Док-во:

1. $\Rightarrow$ (если грамматика L-атрибутовая)

Рекурсивная процедура посещения узла:

- Заходим в узел n
- Просматриваем всех его сыновей m слева направо
- Вычисляем наследуемые атрибуты каждого сына m
- Заходим в соответствующие сыновьям m поддеревья
- Вычисляем синтезируемые атрибуты узла n (когда все сыновья и соответствующие им поддеревья посещены и произведены все возвраты из этих процедур)

Это работает, потому что:

В L-атрибутной грамматике все наследуемые атрибуты зависят только от левых братьев или от отца. Если мы посещаем узел и его сыновей, это значит, что к этому времени наследуемые атрибуты отца уже известны. Когда мы доходим до какого-то узла среди его сыновей, это значит, что всех более левых сыновей мы уже посетили и вернулись оттуда (то есть вычислены их наследуемые и синтезируемые атрибуты), поэтому теперь в случае зависимости от атрибутов левых братьев, мы сможем вычислить атрибуты данного узла. Синтезируемые атрибуты вычисляются после того, как все поддерево, лежащее под данным узлом полностью пройдено, значит у нас есть вся необходимая информация для вычисления синтезируемых атрибутов.

Вычисление атрибутов в L-атрибутной грамматике

II. $\Leftarrow$

Каждый узел дерева при обходе в глубину посещается дважды: при первом посещении он заносится в стек, а при втором удаляется после обхода всех своих потомков. Значит, синтезируемые атрибуты узла можно вычислить только при втором посещении. Кроме того, узлы-братья не находятся в стеке одновременно. При стандартном обходе к моменту первого посещения узла известны атрибуты всех его «левых» братьев, а «правые» братья впервые будут посещены уже после второго посещения узла, т.е., атрибуты узла не могут зависеть от атрибутов правых братьев. Если наследуемые атрибуты вычислять при втором посещении узла, то они не могут зависеть от атрибутов родителя вообще, а если при первом, то наследуемые атрибуты могут зависеть от наследуемых атрибутов родителя. Таким образом, в каждом узле должны реализовываться правила L-атрибутной грамматики.

2. $\Leftarrow$ (если данная процедура позволяет вычислить все атрибуты)

Замечания:

Узлы-братья в стеке одновременно не находятся: если мы прошли какого-то брата, значит мы уже прошли все поддерево, вернулись обратно в этот узел, посчитали все его атрибуты и удалили его из стека.

Если наследуемые атрибуты вычисляются при первом посещении узла, то они могут зависеть только от наследуемых атрибутов родителя, поскольку синтезируемые атрибуты родителя будут вычислены только после посещения всех его сыновей и возвращения в него (родительский).

Схема трансляции

При совмещении семантического анализа с синтаксическим часто используется вариант атрибутивной грамматики, называемый *схемой трансляции*:

$$A \rightarrow [\text{действие}_1]X_1[\text{действие}_2]X_2 \dots [\text{действие}_n]X_n[\text{действие}_{n+1}]$$

Некоторые из действий могут быть пустыми.

Схема трансляции явно задает порядок выполнения семантических действий компилятором при разборе цепочки.

Так, нисходящий анализатор при развертке правила в узле дерева вывода будет обрабатывать правую часть посимвольно слева направо: действия – выполнять, а узлы – создавать и рекурсивно вызывать для них процедуру развертки.

Схема трансляции - это способ записи атрибутивной грамматики

Между символами встраиваются семантические действия, которые происходят в определенный момент

Разворачивать правило в узле = заменять левую часть правила на правую

Схема трансляции

$$A \rightarrow [\text{действие}_1]X_1[\text{действие}_2]X_2 \dots [\text{действие}_n]X_n[\text{действие}_{n+1}]$$

Схема трансляции должна быть построена так, чтобы значения атрибутов, участвующих в действии, были доступны к моменту выполнения действия. Три приводимых ниже ограничения гарантируют доступность значений атрибутов; эти ограничения естественно вытекают из определения L-атрибутивной грамматики.

1. Наследуемые атрибуты X_i вычисляются в i -м действии.
2. i -ое действие не должно обращаться к синтезируемым атрибутам символа X_i и к атрибутам символов X_j , где $j > i$.
3. Синтезируемые атрибуты A вычисляются в $(n+1)$ -м действии.

К моменту выполнения действия должны быть доступны (вычислены) все атрибуты, от которых зависит вычисляемый в данном действии атрибут

Замечания по ограничениям:

1. Атрибуты символа вычисляются в действии, которое находится перед тем, как мы пойдем разворачивать поддерево, соответствующее данному узлу

2. Синтезируемые атрибуты для левой части правила вывода вычисляются в самом последнем действии, которое расположено после последнего символа правой части

Пример: T_EX, вычисление высоты формулы

T_EX рассматривает формулу (либо строку текста) как последовательность блоков. Блоком в данном примере будет либо символ, либо последовательность блоков, заключенная в фигурные скобки. Блоки размещаются либо друг за другом, либо в качестве нижних индексов других блоков (символ подчеркивания является «оператором» нижнего индекса). Например, строка z_zz транслируется в z_zz , строка $z_{\{zz\}}$ – в z_{zz} , а строка $z_{\{z_z\}}$ – в z_{z_z} . Блоки (B) и последовательности блоков (A) имеют синтезируемый атрибут $.ht$ (height) – высоту занимаемого на странице пространства, и наследуемый атрибут $.ps$ (pointsize) – размер шрифта, или *кегель*. Атрибут $sym.h$ означает относительную высоту конкретного символа (определяется по таблице символов).

Последовательность блоков, заключенная в фигурные скобки, является вложенным блоком

Задача: посчитать физическую высоту формулы

A и B - нетерминалы

Атрибут относительной высоты - для терминалов (это запись в таблице символов, лексическое значение)

Пример: атрибутная грамматика

	Правило вывода	Семантические правила
1	$S \rightarrow A$	$A.ps = 10, S.ht = A.ht$
2	$A \rightarrow A_1 B$	$A_1.ps = A.ps, B.ps = A.ps$ $A.ht = \text{MAX}(A_1.ht, B.ht)$
3	$A \rightarrow B$	$B.ps = A.ps, A.ht = B.ht$
4	$B \rightarrow B_1_B_2$	$B_1.ps = B.ps$ $B_2.ps = \text{SHRINK}(B.ps)$ $B.ht = \text{DISPOSE}(B_1.ht, B_2.ht)$
5	$B \rightarrow \{A\}$	$A.ps = B.ps, B.ht = A.ht$
6	$B \rightarrow \text{sym}$	$B.ht = \text{sym.h} * B.ps$

Базовый кегль в примере выбран равным 10. Функция SHRINK вычисляет по кеглю текста кегль его нижнего индекса (индексирование может быть многоступенчатым).

Отдельный символ тоже может быть блоком

Базовый кегль задается откуда-то извне (можно считать константой), далее кегль наследуется, единственное изменение появляется там, где есть нижняя индексация

Кегль передаем сверху вниз по дереву, а высоту вычисляем после этого снизу вверх

В последовательности блоков высота будет максимальной из всех составляющих

Пример: схема трансляции

Функция DISPOSE вычисляет высоту текста с индексом по высотам текста и индекса. Данная грамматика является L-атрибутной, так как единственный наследуемый атрибут *.ps* зависит только от наследуемого атрибута родителя. Ниже приводится соответствующая схема трансляции.

$$\begin{array}{llll}
 S \rightarrow & [A.ps = 10] & A & [S.ht = A.ht] \\
 A \rightarrow & [A_1.ps = A.ps] & A_1 & [B.ps = A.ps] \\
 & & B & [A.ht = \max(A_1.ht, B.ht)] \\
 A \rightarrow & [B.ps = A.ps] & B & [A.ht = B.ht] \\
 B \rightarrow & [B_1.ps = B.ps] & B_1 _ & [B_2.ps = \text{SHRINK}(B.ps)] \\
 & & B_2 & [B.ht = \text{DISPOSE}(B_1.ht, B_2.ht)] \\
 B \rightarrow \{ & [A.ps = B.ps] & A \} & [B.ht = A.ht] \\
 B \rightarrow & & \text{sym} & [B.ht = \text{sym}.h * B.ps]
 \end{array}$$